EE 432

RF Engineering for Telecommunications

Lab # 7

OFDM TRX (Mod/Demodulator)

Name: Mohammad Chahardori (Student ID: 11632753)

Partner: Richard Osorio

Due Date: 12/11/2020

Objectives

In this lab we are supposed to learn how an OFDM modulation and demodulation systems work. we will generate the MATLAB codes for OFDM system and simulate a 16 channel OFDM system with BPSK modulation in all Channels. we also consider a discrete time finite impulse response FIR linear filter as Channel behavior with AWGN noise characteristics to model the behavior of the communication channel. At the end we want to compare the performance of the OFDM modulator and demodulator based on the experimental simulation result and the theoretical results by comparing the BER in each sub-Channel and see how close they are and to what extent the theoretical equations and relations can show the experimental result.

Procedure and Analysis

a): To generate the MATLAB code for simulation of OFDM system We first filled in the missing sections in the MATLAB code provided in this lab, which implements a 16 channel BPSK OFDM simulator. This part of the code computes the channel frequency response vector H. The code for this section and the printout of the numbers in H is as follow:

MATLAB Code for part (a):

```
%OFDM BPSK simulation for Lab7
clear all;
clc;
clf;
sigma2 = 0.5; %Noise variance sigma2 = N0/2.
N = 16; %Number of OFDM channels
h = [1 -0.5 0.3 -0.1 0.4 0.1 -0.05]; %Impulse response
L = length(h) - 1; %Set L, the impulse response length - 1
%Initialize the DFT matrices (see lecture notes and reading)
WN = exp(-j*2*pi/N);
Nrange = (0:N-1)';
Wexponents = Nrange*Nrange'; %matrix of WN exponents
W = WN.^Wexponents; %DFT forward transform
Wi = (1/N)* WN.^(-Wexponents); %DFT inverse transform (you don't actually need to
compute the inverse)
Ws =  (1/sqrt(N))*W; %DFT forward transform scaled by 1/sqrt(N)
Wsi = sqrt(N)*Wi; %DFT inverse scaled by sqrt(N)
%Check the DFT matrices to make sure they multiply to the identity matrix
round(W*Wi)
round(Ws*Wsi)
he = [h zeros(1,N-L-1)]; %Extend h with extra 0s at end to be length N,
%(Easiest by using vector concatenation and the zeros command)
he = he.'; %Convert he from 1xN row vector to Nx1 column vector, using non-conjugate
%Compute channel frequency response H[n] using forward DFT matrix W, he column vector
H = W*he;
%Part a
%Make stem plots of abs(H) and angle(H) vs. Nrange
```

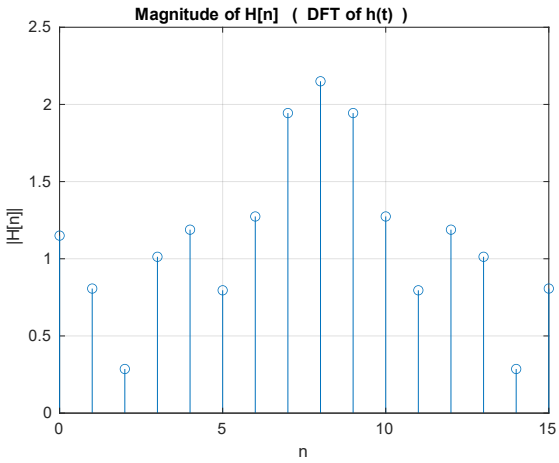
```
figure(1);
stem(Nrange,abs(H));
ylabel('|H[n]|');
xlabel('n');
figure(2);
stem(Nrange,angle(H));
ylabel('angle(H[n])');
xlabel('n');
```

Complex numbers representing H[n] in part (a):

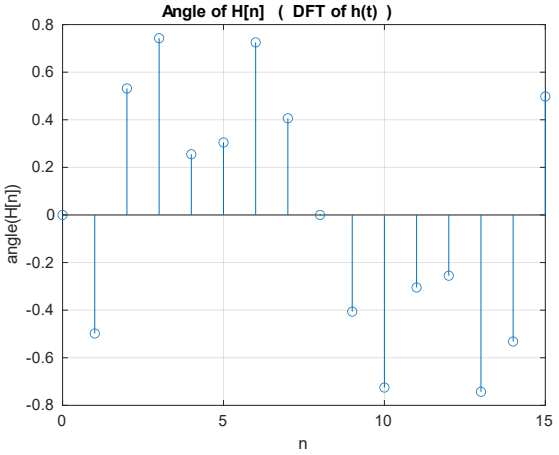
```
H =

   1.1500 + 0.0000i
   0.7090 - 0.3854i
   0.2464 + 0.1450i
   0.7459 + 0.6852i
   1.1500 + 0.3000i
   0.7591 + 0.2387i
   0.9536 + 0.8450i
   1.7860 + 0.7681i
   2.1500 - 0.0000i
   1.7860 - 0.7681i
   0.9536 - 0.8450i
   0.7591 - 0.2387i
   1.1500 - 0.3000i
   0.7459 - 0.6852i
   0.2464 - 0.1450i
   0.7090 + 0.3854i
```

Figure.1 shows the stem plots of abs(H) and angle(H).



(a)



(b)

Figure.1: H[n] plot. (a) abs(H) and (b) angle(H).

(b) In this part we filled the code to calculate and generate vectors: st, s, zr, zrnL, z, zt, sthat, sthatdec, bhat, and errors. In this part the noise vector w = 0, and the number of transmitted vectors n_txd_vectors = 1.
The mentioned vector for part (b) are as follow: (I have printed out the non-conjugate transposed regarding some of these vectors here for better demonstration)

```
b  =  1   0   0   0   0   1   1   1   1   0   1   0   0   1   1   0

st' = 1  -1  -1  -1  -1   1   1   1   1  -1   1  -1  -1   1   1  -1
```

```
s.' =
   1.3536 + 0.5607i
   0.5449 - 0.8155i
  -0.5000 - 0.5000i
   0.1622 - 0.1084i
   0.6464 + 1.5607i
  -0.8155 + 0.1622i
   0.0000 + 0.0000i
  -0.8155 - 0.1622i
   0.6464 - 1.5607i
   0.1622 + 0.1084i
  -0.5000 + 0.5000i
   0.5449 + 0.8155i
   1.3536 - 0.5607i
   0.1084 + 0.5449i
   1.0000 + 0.0000i
   0.1084 - 0.5449i
   1.3536 + 0.5607i
   0.5449 - 0.8155i
  -0.5000 - 0.5000i
   0.1622 - 0.1084i
   0.6464 + 1.5607i
  -0.8155 + 0.1622i
```

```
zr.' =

   1.3536 + 0.5607i
  -0.1319 - 1.0958i
  -0.3664 + 0.0759i
   0.4403 - 0.1591i
   0.9023 + 1.7707i
  -0.6867 - 0.8708i
   0.3723 + 0.0883i
  -1.1371 - 0.3222i
   1.4355 - 0.8574i
  -0.6753 + 1.0664i
  -0.4195 - 0.0680i
   0.4935 + 0.6811i
   1.0919 - 1.4697i
  -0.1846 + 0.9153i
   1.0813 - 0.2333i
  -0.3346 + 0.0454i
   2.2094 + 0.6109i
  -0.0479 - 1.1382i
  -0.0341 + 0.2130i
   0.5783 - 0.4043i
   0.8631 + 1.7162i
  -0.6922 - 0.8435i
   0.3723 + 0.0883i
  -0.3217 - 0.1600i
   0.3813 + 0.6222i
  -0.2697 + 0.2264i
  -0.1139 - 0.0618i
   0.0408 - 0.0081i
```

```
zrnl.' =
   0.3723 + 0.0883i
  -1.1371 - 0.3222i
   1.4355 - 0.8574i
  -0.6753 + 1.0664i
  -0.4195 - 0.0680i
   0.4935 + 0.6811i
   1.0919 - 1.4697i
  -0.1846 + 0.9153i
   1.0813 - 0.2333i
  -0.3346 + 0.0454i
   2.2094 + 0.6109i
  -0.0479 - 1.1382i
  -0.0341 + 0.2130i
   0.5783 - 0.4043i
   0.8631 + 1.7162i
  -0.6922 - 0.8435i
   0.3723 + 0.0883i
  -0.3217 - 0.1600i
   0.3813 + 0.6222i
  -0.2697 + 0.2264i
  -0.1139 - 0.0618i
   0.0408 - 0.0081i
```

```
Z =

   0.3723 + 0.0883i
  -1.1371 - 0.3222i
   1.4355 - 0.8574i
  -0.6753 + 1.0664i
  -0.4195 - 0.0680i
   0.4935 + 0.6811i
   1.0919 - 1.4697i
  -0.1846 + 0.9153i
   1.0813 - 0.2333i
  -0.3346 + 0.0454i
   2.2094 + 0.6109i
  -0.0479 - 1.1382i
  -0.0341 + 0.2130i
   0.5783 - 0.4043i
   0.8631 + 1.7162i
  -0.6922 - 0.8435i
```

```
zt =

   1.1500 - 0.0000i
  -0.7090 + 0.3854i
  -0.2464 - 0.1450i
  -0.7459 - 0.6852i
  -1.1500 - 0.3000i
   0.7591 + 0.2387i
   0.9536 + 0.8450i
   1.7860 + 0.7681i
   2.1500 + 0.0000i
  -1.7860 + 0.7681i
   0.9536 - 0.8450i
  -0.7591 + 0.2387i
  -1.1500 + 0.3000i
   0.7459 - 0.6852i
   0.2464 - 0.1450i
  -0.7090 - 0.3854i
```

```
sthat.' =

   1.0000 - 0.0000i
  -1.0000 - 0.0000i
  -1.0000 + 0.0000i
  -1.0000 - 0.0000i
  -1.0000 - 0.0000i
   1.0000 - 0.0000i
   1.0000 + 0.0000i
   1.0000 + 0.0000i
   1.0000 + 0.0000i
  -1.0000 + 0.0000i
   1.0000 + 0.0000i
  -1.0000 + 0.0000i
  -1.0000 + 0.0000i
   1.0000 + 0.0000i
   1.0000 - 0.0000i
  -1.0000 - 0.0000i


sthatdec =   1  -1  -1  -1  -1   1   1   1   1  -1   1  -1  -1   1   1  -1


bhat     =   1   0   0   0   0   1   1   1   1   0   1   0   0   1   1   0


errors =     0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
```

This is the MATLAB code for part (b):

```
%OFDM BPSK simulation for Lab7
clear all;
clc;
clf;
sigma2 = 0.5; %Noise variance sigma2 = N0/2.
N = 16; %Number of OFDM channels
h = [1 -0.5 0.3 -0.1 0.4 0.1 -0.05]; %Impulse response
L = length(h) - 1; %Set L, the impulse response length - 1
%Initialize the DFT matrices (see lecture notes and reading)
WN = exp(-j*2*pi/N);
Nrange = (0:N-1)';
Wexponents = Nrange*Nrange'; %matrix of WN exponents
W = WN.^Wexponents; %DFT forward transform
Wi = (1/N)* WN.^(-Wexponents); %DFT inverse transform (you don't actually need to
compute the inverse)
Ws =  (1/sqrt(N))*W; %DFT forward transform scaled by 1/sqrt(N)
Wsi = sqrt(N)*Wi; %DFT inverse scaled by sqrt(N)
%Check the DFT matrices to make sure they multiply to the identity matrix
round(W*Wi);
round(Ws*Wsi);
he = [h zeros(1,N-L-1)]; %Extend h with extra 0s at end to be length N, calling
result he
 %(Easiest by using vector concatenation and the zeros command)
he = he.'; %Convert he from 1xN row vector to Nx1 column vector, using non-conjugate
transpose
%Part a
%Compute channel frequency response H[n] using forward DFT matrix W and the he column
vector
H = W*he;
%Part a
%Make stem plots of abs(H) and angle(H) vs. Nrange
figure(1);
stem(Nrange,abs(H));
ylabel('|H[n]|');
xlabel('n');
figure(2);
stem(Nrange,angle(H));
ylabel('angle(H[n])');
xlabel('n');
%Start of parts b and c
%Initialize the cumulative channel error count matrix
cerrors = zeros(1,N);
%Initialize number of symbol vectors to transmit
n_txd_vectors = 1; %For part c you'll set this to 500000 OFDM
%Loop to transmit the symbol vectors
for k = (1:n_txd_vectors)
%For part b:
b = [1 0 0 0 0 1 1 1 1 0 1 0 0 1 1 0];
%For part (c), uncomment the following line
%Generate 16 random bits
```

```
%b = rand(1,N) < 0.5;
%Shift the bits to BPSK symbols: 0->-1, 1->1


for ii = 1:length(b)
    if b(ii)==1
st_temp(ii)=1;
elseif b(ii) == 0;
st_temp(ii)=-1;
end
end
st = st_temp;
%Convert st from 1xN row vector to Nx1 column vector (non-conjugate transpose)
st = st.';
%Take IDFT of st using scaled IDFT matrix Wsi
s = Wsi * st;
%Convert s from column vector to row vector using non-conjugate transpose
s = s.';
%Insert cyclic prefix into s per course notes/reading
%Make sure to insert the cyclic prefix at the START of s,
%so that the first L elements of s are s(N-L+1) to s(N)
%Easy to do this by matrix concatenation using brackets
%Type "help paren" for details. Type "help colon" to find
%out how to pick out selected parts of a vector.
s = [s(N-L+1:N) s];
MCH_LL=length(s);
%Compute received vector zr by convolving h with s, using MATLAB conv
%command.
zr = conv(s,h);
%For part b:
w = zeros(size(zr));
%For part c: Uncomment and complete the line 6 lines below this one.
% Create complex Gaussian noise vector w with total variance sigma2, of same size as
zr
% Make real and imaginary parts have variance sigma2
% Hints: 1) randn produces Gaussian r.v.s with 0 mean and variance 1
% 2) If A is a constant and X an r.v., then var(A*X) = A^2*var(X)


%w = sqrt(sigma2)*randn(size(zr)) + j*sqrt(sigma2)*randn(size(zr)); %Uncomment and
complete for Part c
%w = (sigma2)^2*randn(size(zr)) + j*(sigma2)^2*randn(size(zr)); %Uncomment and
complete for Part c


% Add w to zr, producing zrn
zrn = zr + w;
%Remove cyclic prefix by removing first L samples from zrn, producing zrnL
%Hint: type "help colon" to find out how to pick out selected parts of a vector
zrnL = zrn(L+1:end);
%Keep only first N elements of zrnL; call the result z
z = zrnL(1:N);
```

```
%Convert z to column vector (non-conjugate transpose)
z = z.';
%Take DFT of z using Ws
zt = Ws*z;
%Divide zt by H[n] element by element (use ./)
sthat = zt./H;
%Convert sthat to row vector (non-conjugate transpose)
sthat = sthat.';
%Decode elements of sthat to either -1 or +1 by comparing real part of sthat with 0
%Hint: Use the sign() function
sthatdec = sign(real(sthat));
%Convert sthatdec into bhat by level shifting from -1->0, 1->1
for kk = 1:length(sthatdec)
if sthatdec(kk)==1;
bhat_temp(kk)=1;
elseif sthatdec(kk) == -1;
bhat_temp(kk)=0;
end
end
bhat = bhat_temp;
%XOR b and bhat, using xor command, to produce an integer variable
%called errors, which has a 1 in each bit position that is in error.
errors = xor(b,bhat);
%Update the cumulative total errors in each channel in the cerrors vector
%by adding errors into cerrors
cerrors = cerrors + errors;
end; %end transmitted vectors loop
%End of part b
cerrors;
```

(c). For this part we run the simulation, and calculate the values of the vectors cerrors and cbers. In this part the noise vector w = is calculated from

```
w = sqrt(sigma2)*randn(size(zr)) + j*sqrt(sigma2)*randn(size(zr));
```

and the number of transmitted vectors n_txd_vectors = 500000. Each time the random bits are generated using:

```
b = rand(1,N) < 0.5;
```

and N=16 which is the number of bits in each data vector.

After running the code, we have:

```
cerrors =

26003     63746     171023     37917     23256     65188     17919     1501
553       1492      17738      65171     23241     37755     171596    63585
```

```
cbers =

0.0520    0.1275    0.3420    0.0758    0.0465    0.1304    0.0358    0.0030
0.0011    0.0030    0.0355    0.1303    0.0465    0.0755    0.3432    0.1272
```

(d). In this part we calculate the SNR of each sub-channel using:

snrs = abs(H).^2/(sigma2);

where sigma2 = 0.5 in the calculation of the snrs. The noise vector for each set of transmitted bits is generated using:

w = sqrt(sigma2)*randn(size(zr)) + j*sqrt(sigma2)*randn(size(zr));

in which the sigma2 of the real and imaginary parts of the noise is equal to 0.5 since If A is a constant and X an r.v., then var(A*X) = A^2*var(X) and we know randn command in MATLAB produces Gaussian r.v.s with 0 mean and variance 1.
After running the code, for the SNR related to each 16 sub-channels, we have:

```
snrs' =

2.6450    1.3025    0.1635    2.0518    2.8250    1.2664    3.2465    7.5593
9.2450    7.5593    3.2465    1.2664    2.8250    2.0518    0.1635    1.3025
```

Please note the SNR(1) = 2.6450, SNR(2) = 1.3025, …, SNR(15) = 0.1635, SNR(16) = 1.3025

(e). In this part of the code, we have calculated the BER using theoretical equation and Experimental values and compared them as follow:

cbers'	thbers
0.0513	0.0519
0.1276	0.1269
0.3435	0.3430
0.0754	0.0760
0.0462	0.0464
0.1310	0.1302
0.0352	0.0358
0.0029	0.0030
0.0012	0.0012
0.0029	0.0030
0.0358	0.0358
0.1298	0.1302
0.0463	0.0464
0.0760	0.0760
0.3426	0.3430
0.1267	0.1269

```
ber_avg = 0.0985

BER_ratio = 290.4844
```

Conclusion and discussion:

1- as we can see the theoretical values for bit error rate based on SNR of each sub-channel and using Q function in each sub-Channel is like the experimental value of BER with very good accuracy.
2- As seen the average BER = 0.0985 that means the average BER is somewhat high. that is because the value of added noise is high compared to the signal level and as we can see in snr's in some of the Sub-Channel the SNR are even lower than 1 or a little higher than 1 which is not good for some application.
3- BER_ratio = 290 shows inconsistency between SNR values in different sub-Channel. As I said the SNR in some sub-channel is lower than 1 and in some of sub-channel is around 10. that is because the magnitude of H [n] in different sub-Channel shows too much variation. it also describes without power optimization and proper power allocation among different sub-Channel based on the difference between the SNR and the magnitude of the H[n] we won't be able to achieve high and good power efficiency from this communication Channel.

```
% MATLAB code for part C, d and e.
%OFDM BPSK simulation for Lab7
clear all;
clc;
clf;
sigma2 = 0.5; %Noise variance sigma2 = N0/2.
N = 16; %Number of OFDM channels
h = [1 -0.5 0.3 -0.1 0.4 0.1 -0.05]; %Impulse response
L = length(h) - 1; %Set L, the impulse response length - 1
%Initialize the DFT matrices (see lecture notes and reading)
WN = exp(-j*2*pi/N);
Nrange = (0:N-1)';
Wexponents = Nrange*Nrange'; %matrix of WN exponents
W = WN.^Wexponents; %DFT forward transform
Wi = (1/N)* WN.^(-Wexponents); %DFT inverse transform (you don't actually need to
compute the inverse)
Ws =  (1/sqrt(N))*W; %DFT forward transform scaled by 1/sqrt(N)
Wsi = sqrt(N)*Wi; %DFT inverse scaled by sqrt(N)
%Check the DFT matrices to make sure they multiply to the identity matrix
round(W*Wi);
round(Ws*Wsi);
he = [h zeros(1,N-L-1)]; %Extend h with extra 0s at end to be length N, calling
result he
 %(Easiest by using vector concatenation and the zeros command)
he = he.'; %Convert he from 1xN row vector to Nx1 column vector, using non-conjugate
transpose
%Part a
%Compute channel frequency response H[n] using forward DFT matrix W and the he column
vector
H = W*he;
%Part a
%Make stem plots of abs(H) and angle(H) vs. Nrange
figure(1);
stem(Nrange,abs(H));
ylabel('|H[n]|');
xlabel('n');
figure(2);
stem(Nrange,angle(H));
ylabel('angle(H[n])');
xlabel('n');
%Start of parts b and c
%Initialize the cumulative channel error count matrix
cerrors = zeros(1,N);
%Initialize number of symbol vectors to transmit
n_txd_vectors = 1; %For part c you'll set this to 500000 OFDM
n_txd_vectors = 500000; %For part c you'll set this to 500000 OFDM
%Loop to transmit the symbol vectors
for k = (1:n_txd_vectors)
%For part b:
%b = [1 0 0 0 0 1 1 1 1 0 1 0 0 1 1 0];
%For part (c), uncomment the following line
%Generate 16 random bits
```

```
b = rand(1,N) < 0.5;
%Shift the bits to BPSK symbols: 0->-1, 1->1


for ii = 1:length(b)
    if b(ii)==1
        st_temp(ii)=1;
    elseif b(ii) == 0;
        st_temp(ii)=-1;
    end
end
st = st_temp;
%Convert st from 1xN row vector to Nx1 column vector (non-conjugate transpose)
st = st.';
%Take IDFT of st using scaled IDFT matrix Wsi
s = Wsi * st;
%Convert s from column vector to row vector using non-conjugate transpose
s = s.';
%Insert cyclic prefix into s per course notes/reading
%Make sure to insert the cyclic prefix at the START of s,
%so that the first L elements of s are s(N-L+1) to s(N)
%Easy to do this by matrix concatenation using brackets
%Type "help paren" for details. Type "help colon" to find
%out how to pick out selected parts of a vector.
s = [s(N-L+1:N) s];
MCH_LL=length(s);
%Compute received vector zr by convolving h with s, using MATLAB conv
%command.
zr = conv(s,h);
%For part b:
%w = zeros(size(zr));
%For part c: Uncomment and complete the line 6 lines below this one.
% Create complex Gaussian noise vector w with total variance sigma2, of same size as
zr
% Make real and imaginary parts have variance sigma2
% Hints: 1) randn produces Gaussian r.v.s with 0 mean and variance 1
% 2) If A is a constant and X an r.v., then var(A*X) = A^2*var(X)


w = sqrt(sigma2)*randn(size(zr)) + j*sqrt(sigma2)*randn(size(zr)); %Uncomment and
complete for Part c
%w = (sigma2)^2*randn(size(zr)) + j*(sigma2)^2*randn(size(zr)); %Uncomment and
complete for Part c


% Add w to zr, producing zrn
zrn = zr + w;
%Remove cyclic prefix by removing first L samples from zrn, producing zrnL
%Hint: type "help colon" to find out how to pick out selected parts of a vector
zrnL = zrn(L+1:end);
%Keep only first N elements of zrnL; call the result z
z = zrnL(1:N);
```

```
%Convert z to column vector (non-conjugate transpose)
z = z.';
%Take DFT of z using Ws
zt = Ws*z;
%Divide zt by H[n] element by element (use ./)
sthat = zt./H;
%Convert sthat to row vector (non-conjugate transpose)
sthat = sthat.';
%Decode elements of sthat to either -1 or +1 by comparing real part of sthat with 0
%Hint: Use the sign() function
sthatdec = sign(real(sthat));
%Convert sthatdec into bhat by level shifting from -1->0, 1->1
for kk = 1:length(sthatdec)
if sthatdec(kk)==1;
bhat_temp(kk)=1;
elseif sthatdec(kk) == -1;
bhat_temp(kk)=0;
end
end
bhat = bhat_temp;
%XOR b and bhat, using xor command, to produce an integer variable
%called errors, which has a 1 in each bit position that is in error.
errors = xor(b,bhat);
%Update the cumulative total errors in each channel in the cerrors vector
%by adding errors into cerrors
cerrors = cerrors + errors;
end; %end transmitted vectors loop
%End of part b
cerrors;


%Compute experimental channel bit error rates for each channel in the cbers vector
cbers = cerrors/n_txd_vectors;
%Part (d)
%Compute SNRs for each subchannel per course notes, using H and sigma2
%Note that these SNRs should be ratios, not in dB.
snrs = abs(H).^2/(sigma2);
%Part (e)
%Compute theoretical BERs (use BPSK formula); they should agree with cbers
thbers = Q(sqrt(snrs));
%Print out comparison of experimental and theoretical BERs
[cbers' thbers]
%Compute average experimental BER over all channels
ber_avg = sum(thbers)/length(thbers)
%Compute ratio of largest to smallest experimental channel BER
%Hint: use max() and min() functions
BER_ratio = max(thbers)/min(thbers)
```

Please See this part related to Lab 9

==========

==========

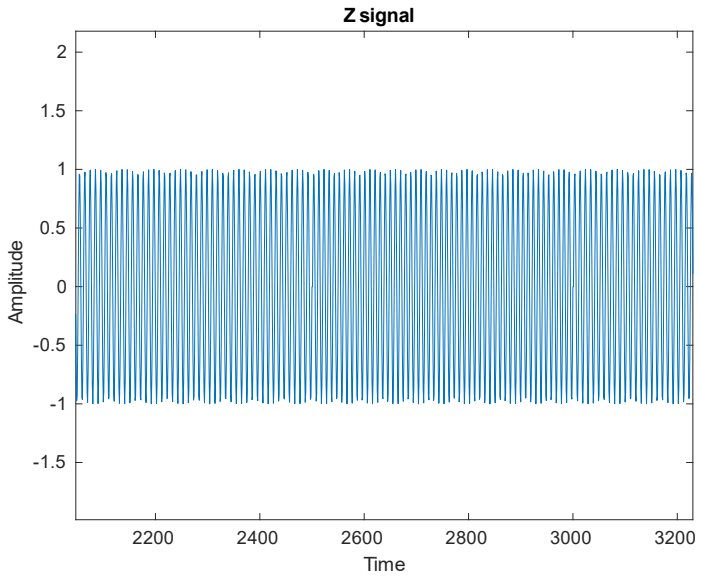
==========

Dear Professor,

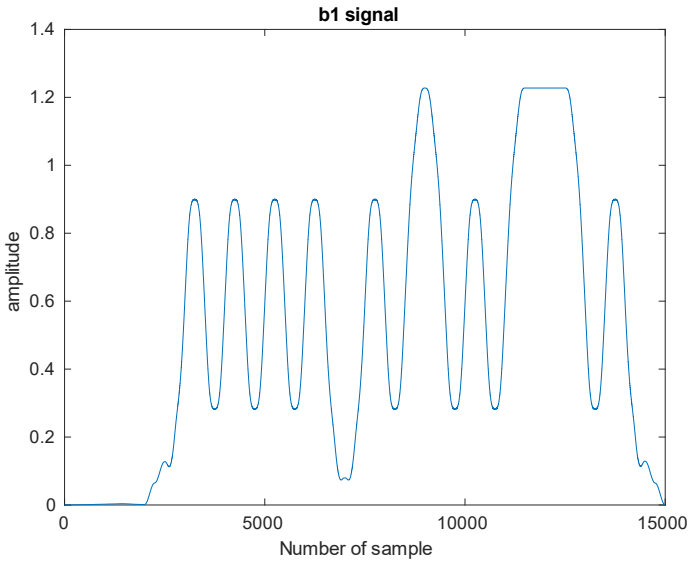
Based on the instruction for lab 9 and lab 10 I just put the Matlab code for lab 9 in the lab report for 9 and 10. In fact I remember you said you want just one report for lab9 and 10 and since in the lab instruction for lab 9 you just asked us to put the matlab code I just put the matlab code without the figures, However today, in the last session of the class you said you give two separate score to lab 9 and 10 so I think you expected us to have two reports including figures of 9 and figures and description of 10.

I just run the code provided in report for lab 9 and I generated the figure for lab 9 here. I don't know if you accept them or not however I hope you understand I have done lab 9 completely in time and due to a misunderstanding I did not add the figures of lab 9 to the lab reports.

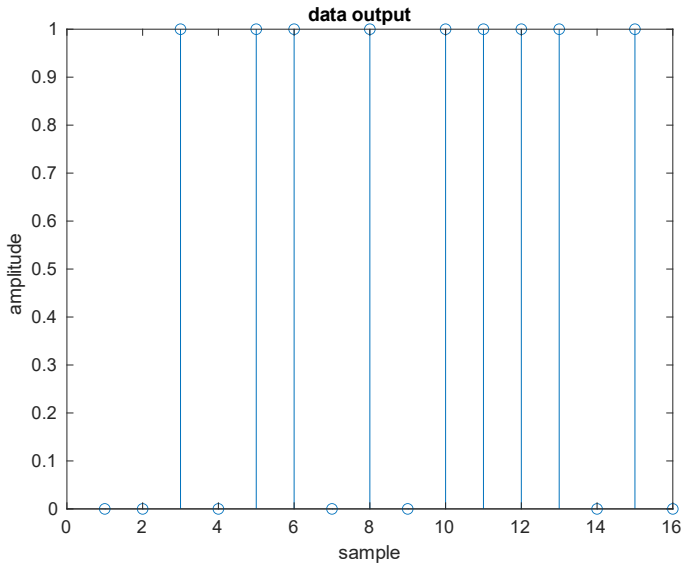
Plot of signal z:



Signal at node b1:



The decoded output data:




```
Input data:

Data  = [ s0 s0 s1 s0 s1 s1 s0 s1 s0 s1 s1 s1 s1 s0 s1 s0 ];

data_output =

   0     0     1     0     1     1     0     1     0     1     1     1     1     0     1     0
```

➔ Input and Output are exactly the same.